

题目 2: NEP 势函数 OpenMP 并行优化代码修改说明

2026 年 6 月 6 日

1 修改目标

本次修改针对 ABACUS 分子动力学模块中的 NEP CPU 势函数计算路径，目标是完成题目 2 所要求的 OpenMP 多线程加速，并保证多线程计算与单线程计算结果一致。

主要修改文件如下：

- `potential/ml/nep/nep_cpu.cpp`: 实现 NEP 力计算路径的 OpenMP 并行；
- `test/nep_omp_test.cpp`: 增加单线程和多线程结果一致性测试；
- `test/CMakeLists.txt`: 注册新增的 NEP OpenMP 单元测试；
- `tools/nep_omp_sweep.cpp`: 实现不同原子数和线程数的测试程序；
- `tools/run_problem2_nep_sweep.sh`: 编译并运行完整测试，保存 CSV 结果。

2 原始代码的问题

2.1 可以直接并行的部分

NEP 代码中的邻居表构建和描述符计算已经以中心原子 n_1 为循环变量进行并行。不同线程主要读取共享输入，并写入与 n_1 对应的独立输出位置，因此不存在明显写冲突。

2.2 力计算中的数据竞争

力计算不能简单地在最外层循环前加入 `#pragma omp parallel for`。对于一对原子 n_1 和 n_2 ，计算结果会同时更新两个原子的受力：

$$F_{n_1} += f_{12}, \quad F_{n_2} -= f_{12}.$$

虽然不同线程负责不同的中心原子 n_1 ，但是它们可能同时把同一个原子当作邻居 n_2 。因此多个线程会同时修改相同的 `force` 和 `virial` 数组元素，形成数据竞争，导致结果不稳定或错误。

原始代码中的典型共享写操作如下：

```
1 g_fx[n1] += f12[0];  
2 g_fx[n2] -= f12[0];  
3 g_virial[n2 + 0 * N] -= r12[0] * f12[0];
```

3 并行设计方案

3.1 线程私有缓冲区

本次修改采用线程私有缓冲区加最终归约的方案。每个 OpenMP 线程分别创建自己的受力和维里数组：

```
1 std::vector<double> fx_private(N, 0.0);
2 std::vector<double> fy_private(N, 0.0);
3 std::vector<double> fz_private(N, 0.0);
4 std::vector<double> virial_private(N * 9, 0.0);
```

在线程执行邻居循环时，只修改当前线程的私有数组，不修改共享输出数组。完成并行循环之后，再将各线程的私有结果累加到最终结果。

这种设计具有以下优点：

- 避免内层邻居循环中的数据竞争；
- 不需要对每一次力更新使用原子操作；
- 保持原有力和维里计算公式不变；
- 单线程和多线程路径使用相同的核心计算逻辑。

3.2 统一归约辅助函数

在 `nep_cpu.cpp` 的匿名命名空间中新增了 `reduce_private_buffer` 函数：

```
1 void reduce_private_buffer(
2     const std::vector<double>& source,
3     double* target)
4 {
5     if (target == nullptr || source.empty()) {
6         return;
7     }
8
9     for (std::size_t i = 0; i < source.size(); ++i) {
10         target[i] += source[i];
11     }
12 }
```

该函数负责把一个线程的私有数组累加到共享输出数组中，并处理空指针和空数组情况。

4 三个力计算函数的具体修改

4.1 径向力计算

修改函数：

```
1 find_force_radial_small_box
```

函数外层加入 OpenMP 并行区域，每个线程建立私有 `fx`、`fy`、`fz` 和 `virial` 数组。中心原子循环通过 `omp for schedule(static)` 分配给各线程：

```
1 #pragma omp parallel
2 {
3     std::vector<double> fx_private(N, 0.0);
4     ...
5
6     #pragma omp for schedule(static)
7     for (int n1 = 0; n1 < N; ++n1) {
8         ...
9     }
10 }
```

原本对共享数组 `g_fx`、`g_fy`、`g_fz` 和 `g_virial` 的写操作，改为写入线程私有指针 `fx`、`fy`、`fz` 和 `virial`。

4.2 角向力计算

修改函数：

```
1 find_force_angular_small_box
```

角向力计算采用与径向力相同的线程私有缓冲区方案。每个线程还会在栈上保存中心原子对应的 `Fp` 和 `sum_fxxyz` 临时数据，因此这些变量天然为线程私有，不需要额外同步。

4.3 ZBL 短程排斥力计算

修改函数：

```
1 find_force_ZBL_small_box
```

ZBL 路径除了受力和维里，还会更新每个原子的势能 `g_pe`。因此该函数额外创建 `pe_private`：

```
1 std::vector<double> pe_private(N, 0.0);
```

并行循环结束后，力、维里和势能私有数组统一归约到共享输出。

4.4 归约过程

每个并行区域结束前使用临界区完成归约：

```
1 #pragma omp critical
2 {
3     reduce_private_buffer(fx_private, g_fx);
4     reduce_private_buffer(fy_private, g_fy);
5     reduce_private_buffer(fz_private, g_fz);
```

```

6   reduce_private_buffer(virial_private, g_virial);
7 }

```

临界区只用于最终数组累加，不位于计算量较大的邻居循环内部。因此相比在每次原子对更新时使用 `omp atomic`，同步次数显著减少。

5 兼容单线程和非 OpenMP 编译

所有 OpenMP 代码都由 `_OPENMP` 宏保护：

```

1  #if defined(_OPENMP)
2      // OpenMP parallel implementation
3  #else
4      // Original serial pointers
5  #endif

```

当编译器没有启用 OpenMP 时，代码继续直接使用原始输出指针，保持串行行为。因此本次修改不会破坏非 OpenMP 构建。

6 正确性测试修改

6.1 新增单元测试

新增文件 `test/nep_omp_test.cpp`。测试使用项目中已有的 `nep_hfo2.txt` 模型，构造一个确定性的六原子 Hf/O 体系。

测试步骤如下：

1. 调用 `omp_set_num_threads(1)`，获得单线程参考结果；
2. 调用 `omp_set_num_threads(4)`，获得多线程结果；
3. 分别比较势能、受力和维里数组。

测试容差设置为：

$$|\Delta E| \leq 10^{-12}, \quad |\Delta F| \leq 10^{-10}, \quad |\Delta V| \leq 10^{-10}.$$

6.2 CMake 测试接入

在 `test/CMakeLists.txt` 中新增测试目标 `MODULE_MD_nep_omp`，并编译 NEP 所需源文件。模型文件路径通过 `NEP_TEST_MODEL_FILE` 编译宏传入，使测试不依赖运行时工作目录。

7 性能测试脚本修改

7.1 测试程序

`tools/nep_omp_sweep.cpp` 实现以下功能：

- 根据指定原子数生成规则 Hf/O 体系；
- 支持多个原子规模和多个 OpenMP 线程数；
- 使用单线程结果作为正确性参考；
- 重复调用 `NEP::compute` 测量平均运行时间；
- 输出时间、加速比和最大绝对误差。

模型加载在计时循环外完成，因此测量结果主要反映 NEP 计算过程，而不是文件读取时间。

7.2 运行脚本

`tools/run_problem2_nep_sweep.sh` 自动完成：

1. 查找支持 OpenMP 的 C++ 编译器；
2. 使用 `-O2 -fopenmp` 编译测试程序；
3. 执行不同规模和线程数组合；
4. 使用 `tee` 保存 CSV 结果。

默认运行命令为：

```
1 cd source/source_md
2 tools/run_problem2_nep_sweep.sh
```

8 修改结果

完整测试覆盖了 64、216 和 512 个原子的体系，以及 1、2 和 4 个 OpenMP 线程。结果显示：

- 所有多线程势能结果与单线程结果完全一致；
- 受力最大误差约为 10^{-15} ；
- 维里最大误差约为 10^{-14} ；
- 4 线程运行获得约 2.94 至 3.55 倍加速。

这些误差属于浮点数累加顺序变化引起的正常舍入误差，说明修改后的代码不存在可观察的线程安全问题。

9 开销与可继续优化方向

当前方案的主要额外开销是线程私有数组，其空间复杂度约为：

$$O(TN),$$

其中 T 为线程数， N 为原子数。

此外，最终归约使用临界区，线程数很高时可能成为瓶颈。后续可以继续研究：

- 将数组分块后进行分层归约；
- 复用线程私有缓冲区，减少每次计算时的内存分配；
- 使用支持数组归约的 OpenMP 版本；
- 结合 MPI 空间分解，减少单进程中的原子规模。

10 总结

本次修改没有改变 NEP 势函数的物理计算公式，而是重新组织了力、维里和 ZBL 势能的写入方式。通过线程私有缓冲区消除数据竞争，再通过最终归约合并结果，实现了安全且具有明显加速效果的 OpenMP 力计算路径。同时新增了单元测试、性能扫描程序和自动运行脚本，使正确性和性能结果均可重复验证。